Mixed-Criticality Scheduling in Real-Time Systems

Sayed Galib Hossen Rizvi
Matriculation Number: 2200588
B.Eng. Electronics Engineering
Real-Time Systems Seminar
Hamm-Lippstadt University of Applied Sciences

***Abstract*—Real-time systems are widely used in safety-critical applications such as avionics, automotive control, industrial automation, and medical devices. In these systems, the correctness of a computation depends not only on the logical result but also on the time at which the result is produced. Modern embedded platforms often execute multiple tasks with different levels of importance on shared hardware resources. While some tasks are safety-critical and must always meet their deadlines, others can tolerate occasional delays or degraded service.**

Mixed-criticality scheduling has emerged as an important approach for managing such systems. It allows tasks with different assurance and safety requirements to coexist while guaranteeing the timing requirements of high-criticality tasks during overload situations. This paper introduces the fundamentals of real-time systems and mixed-criticality systems and presents the scheduling model proposed by Vestal. Furthermore, the paper discusses execution time assurance, mode switching behavior, and a case study derived from the original work. The objective is to provide an overview of how mixed-criticality scheduling improves the predictability and safety of modern embedded real-time systems.

***Index Terms*—Real-Time Systems, Mixed-Criticality Scheduling, Embedded Systems, Task Scheduling, Safety-Critical Systems, Vestal Model**

I. Introduction

Real-time systems are significant components of several technologies in use today, ranging from cars and planes, to medical equipment, and automation systems. In contrast to traditional computer systems, the requirement for a real-time system is not just to ensure that the right outcome is obtained, but to get the right outcome within a given time constraint. The correctness of a real-time system is determined by the correctness of its computation and the time within which the result is obtained [1]. With the increasing complexity of embedded systems, more functions with various degrees of criticality have been found to be sharing a single hardware architecture. The operation of a plane can involve flight control software, navigation, communication, and passenger entertainment among other activities all using the same computing resource. It is noteworthy that all these functions are not equally critical to the safety of the system. The traditional method of real-time scheduling is usually based on one WCET for every function. Software tasks frequently demand varying degrees of assurance, leading to the need for multiple time execution estimates based on different confidence levels of the analysis performed [2]. The selection of just the most conservative estimate leads to wastage of processing power. This problem has been solved by proposing the theory of mixed-criticality scheduling. In mixed-criticality scheduling systems, each task is assigned a particular criticality level based on its importance. The scheduling scheme presented by Vestal assumes that tasks have several execution times and criticalities such that very critical tasks can continue being guaranteed in terms of timing even in the case of overload [2]. In such a situation, less critical tasks can get lower priority or be suspended for some time in order to ensure the safety and reliability of higher critical tasks. The aim of this paper is to give a brief overview of mixed-criticality scheduling in real-time systems. At first, the basic concepts related to real-time systems are described. Then, the notion of mixed-criticality systems and the scheduling scheme presented by Vestal are introduced.

II. Real-Time Systems

Real-time systems refer to computer systems where the correct behavior of the computer system does not depend only on the correctness of the output produced but also depends on the time when that output is generated [1]. In contrast to traditional computer systems whose performance is normally assessed based on throughput and response time, real-time systems should meet certain timing constraints for proper functioning. The main feature of real-time systems is the capability of responding to the events from outside within certain deadlines. The deadlines are the latest possible moments in time by which the execution of a given task must be completed. If a task fails to complete its execution before the deadline expires, the system may face either performance problems or failures in the worst case [3]. Real-time systems usually work within the context of safety-critical or mission-critical systems. Typical examples of real-time systems include airplane flight control systems, car braking systems, automation machinery, railway signal systems, and medical monitoring machines. In such cases, it is equally important that the task be completed on time as that it be completed correctly. A correct output which comes out after an unnecessary delay can be regarded as useless and sometimes dangerous. Real-time system tasks are usually categorized according to their activation characteristics. Periodic tasks activate at a fixed frequency and are mostly used in monitoring and control operations. Aperiodic tasks have unpredictable activation characteristics and are usually activated due to certain events outside the system, like user interaction and emergencies. The job of the scheduler is to

ensure that both kinds of tasks are scheduled. Predictability is another essential characteristic of real-time systems. They are created to operate with predictable and deterministic behavior given certain operating conditions. Predictability provides a way of analyzing the system beforehand and ensuring that all critical tasks will satisfy their deadlines. Therefore, scheduling algorithms and execution-time analysis become very crucial in the design process.

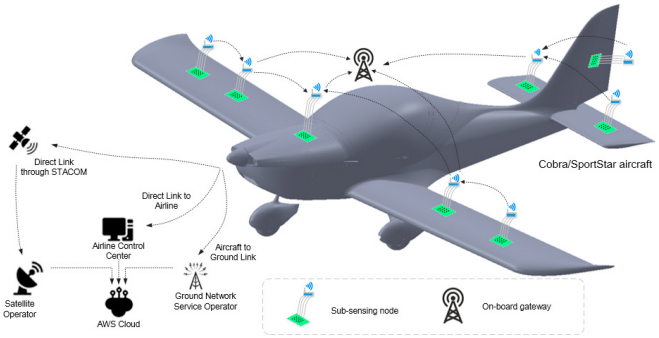

Fig. 1. Example of a modern aircraft embedded and communication system architecture.

III. Mixed-Criticality Systems

A. Criticality Levels

The mixed-criticality systems have tasks with varying levels of criticality and safety constraints. Criticality defines the level of assurance needed for a particular task and consequences of the failure to meet a specific deadline or proper execution of a task. In the case of mixed-criticality systems, there is a classification of tasks based on the level of required assurance for the proper functioning and timely execution of those tasks [2]. For instance, the task of an aircraft flight control is highly critical while display or entertainment task has low safety requirement. Hence, both tasks should not get the same degree of guarantee during the processor overload state. Timing and criticality parameters define the representation of a task in real-time. Task period is responsible for the frequency of task execution, deadline for the moment of the task completion, and the criticality level shows its required level of assurance. In the basic model of a mixed-criticality system, tasks are classified in two classes: low criticality (LO) and high criticality (HI). High-criticality tasks represent the tasks responsible for important/safety-critical functions of the system. On the contrary, the low-criticality tasks have relatively lower importance for system safety.

$$L_i \in \{LO, HI\} \tag{1}$$

Here, L_i represents the criticality level of task i. The notation shows that a task can be classified as either LO-criticality or HI-criticality in the dual-criticality model. The Vestal model also defined various estimates of execution time for each task. It is worth noting that several estimates of worst-case execution time can be estimated for a single task; the higher the assurance level, the higher the probability that the estimation will be conservative [2]. LO-WCET is estimated with standard operational assumptions, while HI-WCET is the safest estimate of execution time in conditions of high criticality. As a result, the HI-WCET is normally not less than LO-WCET. Criticality and priority are two separate notions. The priority indicates which task should run first; criticality is the degree of assurance needed and a potential risk associated with the failure of a task [4]. A low-criticality task may have a high scheduling priority due to proximity of its deadline, but it remains a low-criticality task nevertheless. Many mixed-criticality scheduling models imply two types of criticality – LO and HI – but systems with more than two criticality levels could be considered [4]. In case of processor overload, the criticality levels are used by the scheduler to protect essential tasks and suspend others.

$$C_i(HI) \geq C_i(LO) \tag{2}$$

Here, $C_i(LO)$ represents the LO-WCET estimate of task i, while $C_i(HI)$ represents its HI-WCET estimate. This relation shows that the HI-WCET is greater than or equal to the LO-WCET because it uses more conservative execution-time assumptions and provides a higher level of assurance.e

B. Motivation and Challenges

The increasing complexity of embedded systems is among the primary reasons for introducing the mixed-criticality scheduling. The modern embedded systems unite various software functions in order to lower costs of hardware resources, decrease its size, weight, and energy consumption. On the other hand, various software functions can be characterized by different safety and time requirements. The joint operation of the functions of different levels of criticality in one computing system leads to a number of problems associated with their scheduling. First of all, high-criticality functions need more guarantees in terms of the time than low-criticality tasks do [4]. In traditional scheduling algorithms, only one conservative WCET value of each task is used. Though this approach provides guaranteed time limits, sometimes it overestimates the required processor time. The second problem is related to the uncertainty of execution time. It depends on input data, path of the program, cache effects, etc. In order to obtain a guaranteed WCET estimation, it is necessary to conduct more analysis and testing, especially for safety-critical tasks [2]. Thus, the use of a conservative estimation for each function leads to inefficient processor usage. Processor overload is another key issue. Overload happens if the process takes longer than predicted or if the overall load on the processor increases above the threshold value. In this situation, it will become impossible to ensure all the tasks meet their deadlines. This issue is solved by the mixed-criticality scheduling technique, where tasks that have higher levels of criticality are protected, whereas those with lower criticality are served less. This approach, however, presents some kind of tradeoff between safety and efficiency. The more processor time we reserve, the better the timeliness of critical tasks will be guaranteed, but the fewer resources there will be available for others. The ultimate

goal of the mixed-criticality scheduling is to offer enough guarantees to high-criticality tasks using efficient allocation of processing resources [4].

IV. Vestal's Mixed-Criticality Model

A. Execution Time Assurance

Execution-time prediction is an essential part of mixed criticality systems since real-time scheduling requires the knowledge about execution time that is predictable to some extent. The Worst-Case Execution Time, or WCET, is the upper bound estimate for execution time of a particular task on the specific hardware platform. An accurate estimation of WCET is a challenging task because execution time depends on the program path, behavior of cache, architecture of processor and even the values of the input data. Various WCET techniques have different degree of reliability of execution time estimation [5]. Measurement-based techniques monitor the execution time of tasks during the tests and take the maximum value as the estimate. These techniques are relatively simple to implement, however, it is hard to ensure that all possible program paths were analyzed. Static techniques analyze the program itself and all possible paths of execution not depending on the measurements during runtime. In the context of mixed-criticality systems, the confidence needed to meet the execution-time estimate requirement will depend on the criticality level of the task. That is, the higher the criticality of the software, the greater the degree of confidence that the estimate will be met in terms of WCET [6]. Thus, different techniques will yield different WCETs for the same task. For instance, a low-confidence estimate could be based on normal testing of the software, whereas a high-confidence estimate will be based on deeper program analysis and assumptions. The high-confidence estimate would normally have a higher WCET since more possible execution cases have been considered. Having different estimates helps in mixed-criticality scheduling, since it represents different degrees of execution-time confidence for the same task [7]. In normal running of the task, less conservative estimates could be utilized in order to increase processor usage. If need arises for higher assurance, then a more conservative estimate could be used.

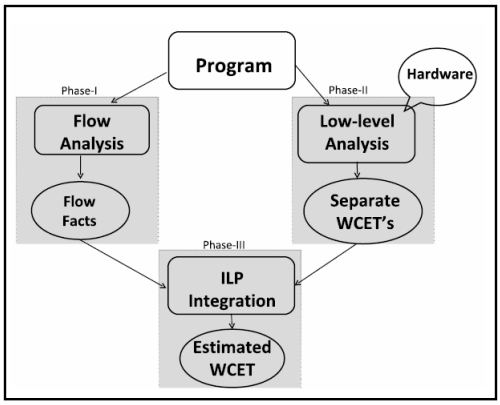

Fig. 2. General process for static worst-case execution time analysis.

B. Scheduling Behaviour

Mixed criticality scheduling alters the allocation of processing times of tasks based on their criticality and requirements for execution. In normal operations, tasks of both low and high criticality are permitted to be executed. The scheduler employs its scheduling policy to try and complete all tasks before their deadlines. In order for the decision on scheduling to be made, not only should the requirements regarding timing but also the criticality of a task be taken into account when applying different degrees of execution-time guarantees [6]. High criticality task requires more conservative execution-time assumptions while other tasks could have processor times allocated based on non-conservative assumptions. The processor demand of a task can be expressed using its execution time and period. The utilization of task τ_i is defined as

$$U_i = \frac{C_i}{T_i} \tag{3}$$

where C_i is the execution time of task i and T_i is its period. The total processor utilization for n tasks is

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i}. \tag{4}$$

Here, the symbol $\sum$ represents the summation of the utilization of all tasks. A larger utilization value indicates that a greater proportion of the available processor capacity is required by the task set. A mixed-criticality system may experience variability in processor demand, depending on the WCET estimation applied to individual tasks. Consequently, the schedulability of a set of mixed-criticality tasks is dependent not only on periods and deadlines of the tasks, but on the execution-time estimates, which correspond to various criticality levels [8]. If a highly critical task needs more execution time than estimated in its normal operations, providing the identical processor service to all tasks might cause the deadlines of critical tasks to be missed. It means that the scheduler should be able to modify its behavior when the execution-time conditions have changed. Timing of highly critical tasks is ensured with increased guarantees, while the execution of low-criticality tasks may be limited. This change in the behavior of the scheduler becomes the basis of mode switching discussed below.

C. Mode Switching

Mode switching constitutes the primary runtime behaviour of mixed-criticality scheduling. The system operates in LO mode by default, in which both low- and high-criticality tasks are allowed to execute. In LO mode, the tasks are scheduled using the LO-WCET estimates, aiming at achieving efficient scheduling of the whole task set.

The execution of high-criticality tasks is monitored by the scheduler at runtime. When a high-criticality task exceeds its estimated LO-WCET in its execution, the execution assumptions of LO mode are deemed insufficient [6]. This phenomenon is referred to as execution-time overrun and can

imply potential processor overload. Under this condition, the system may switch from LO to HI mode.

In HI mode, high-criticality tasks are subject to their more conservative HI-WCET estimates. There should be sufficient processor execution time to guarantee the deadlines of those tasks. To achieve additional processor time for this purpose, some low-criticality tasks might be postponed, given reduced service, or entirely suspended. Decreasing low-criticality workload creates more processor time for the tasks with more robust timing guarantees [8].

Thus, the mode switch means the compromise between the functionality of the system and its safety. Some of the functions in the system may cease being available for a certain period; however, critical functions will still benefit from more reliable timing guarantees. Whether the system remains in HI mode or goes back to LO mode depends on the particular scheduling model [4].

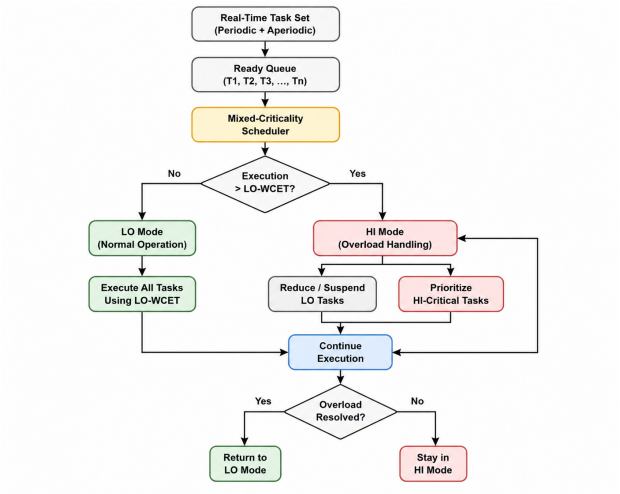

Fig. 3. Mode-switching behaviour in a mixed-criticality scheduling system.

D. Limitations and Trade-offs

Despite the fact that mixed-criticality scheduling enhances the security of safety-critical tasks, there are still some drawbacks to consider. First, it is hard to estimate WCET accurately, and conservative estimates of execution times decrease processor usage. Switching modes may also lead to additional runtime overhead. To protect high-criticality tasks from being affected by overload, low-criticality tasks may be delayed, throttled, and even suspended [4]. Thus, a compromise should be found between the system's safety and the availability of less important functionality. Advanced mixed-criticality algorithms allow for better resource allocation but make implementation and verification more complicated [7]. In other words, mixed-criticality scheduling implies finding a trade-off between timing, efficiency, and overall system performance.

V. Case Study

A. Example from Vestal's RTSS 2007 Paper

The evaluation performed by Vestal was practical and it relied on processor workload data collected from three industrial avionics systems. Different application teams had created those systems which run under different platform configurations. In these workloads, tasks with known execution times and execution time budgets were included. The experimental findings indicated that allocated execution time budgets were larger than the longest execution times observed during experiments [2]. While having more execution time budget enhances the timing assurances, it might also mean that there is some extra processing capacity reserved which is otherwise unused. To perform an evaluation of mixed criticality scheduling, mixed criticality workloads were formed using the available data of the industrial avionics. Traditional deadline monotonic scheduling was then compared to multi criticality analysis and period transformation. Critical scaling factor, Δ^*, was used as the primary measure. Critical scaling factor is the largest factor by which tasks' execution times can be scaled up while keeping the task set schedulable. It means that larger Δ^* shows that the task set can withstand a larger execution demand while remaining schedulable. There have been observed 11%, 42% and 59% improvements for the three workloads analyzed. The best performance was achieved in case of Workload 3. It means that in some cases mixed criticality analysis is able to greatly improve schedulability of certain industrial task sets. Workload characteristics, comparison methods, definition of Δ^* and improvements of 11%, 42%, 59% were described by Vestal in his original RTSS paper [2].

B. Discussion of Results

According to the results of the experiment, the mixed-criticality analysis has provided a positive improvement in the schedulability of all three avionic workloads. Nevertheless, the degree of this improvement was not the same in all cases. Thus, the effectiveness of the mixed-criticality scheduling largely depends on the timing properties and margins of execution time of the task set. So, for the first workload, the critical scaling factor became higher by 11% (from 1.08 to 1.20). The second workload shows that the period transformation combined with multi-criticality analysis increases the factor from 1.24 to 1.76, thus providing a 42% increase. The highest increase has been achieved for the third workload (from 1.07 to 1.70), which corresponds to 59% [2].

This proves that the mixed-criticality scheduling allows improving the utilization of processors by taking advantage of the difference between normal and high-assurance execution-time estimates [**?**]. Nonetheless, the degree of improvement is very workload-dependent [9]. The differences between the three workloads show that an increased execution-time margin cannot lead to increased schedulability gain. Workload 2 had an increased average margin of measured against allocated execution time, whereas the greatest gain was obtained by Workload 3. It thus follows that task periods, deadlines,

priorities, and criticality distribution are additional factors which can impact the efficiency of mixed-criticality analysis [9]. Increased values of Δ^* imply that the workload has an increased tolerance to the proportion of increase in execution requirements without becoming unschedulable. In practice, this can mean that more functionality can be run by the processing platform within the specified time constraints. It is especially significant in the case of embedded systems, since the processing capabilities and power consumption are limited, in addition to size and cost of the hardware. Nevertheless, the above results were obtained for only three workloads from avionics applications, and do not ensure similar gains for all other real-time systems. The experiments only illustrate the possibilities offered by mixed-criticality scheduling, and show that the gain highly depends on the task set characteristics.

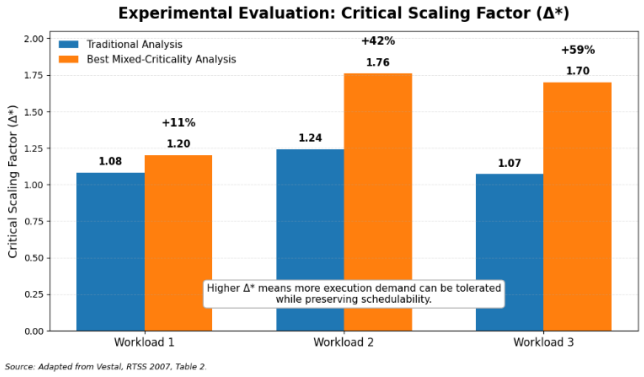

Fig. 4. Comparison of the critical scaling factor for traditional and mixed-criticality analysis, adapted from Vestal [2].

VI. Conclusion

Mixed-criticality scheduling provides an effective approach for managing tasks with different criticality levels on the same computing platform. Different from the classic real-time scheduling, it takes into account the criticality of tasks and permits different estimates of execution time to correspond to different levels of assurance. This enables the system to utilize processor resources better during regular operation but to ensure safety-critical functions. The usage of LO and HI operating modes enables the system to react in case of exceeding the estimated LO-WCET by a high-criticality task. In such a situation, the system switches to HI mode, providing strong guarantees for high-criticality tasks. Tasks with lower criticality can be delayed, limited, or even suspended in order to satisfy some deadlines. Thus, mode switching serves as an adaptive technique of response to processor overload. As for Vestal's industrial avionics workloads, the evaluation showed that mixed-criticality analysis improves the schedulability of tasks compared to traditional techniques. The mentioned improvement of 11%, 42%, and 59% proves that the advantage is essential but is highly dependent on the workload characteristics. However, mixed-criticality scheduling also faces several difficulties like accurate WCET estimation, mode switching overhead, and decreased service of low-criticality tasks. Thus, in general, the technique is a good compromise between timing assurance, processor utilization, and system safety.

Acknowledgment

The author would like to express sincere gratitude to Prof. Dr. Stefan Henkler of Hamm-Lippstadt University of Applied Sciences for his guidance, lectures, and valuable insights into the field of real-time systems, which contributed significantly to the preparation of this seminar paper. "'

References

[1] H. Kopetz, *Real-Time Systems: Design Principles for Distributed Embedded Applications*, 2nd ed. Springer, 2011.

[2] S. Vestal, "Preemptive scheduling of multi-criticality systems with varying degrees of execution time assurance," in *Proceedings of the 28th IEEE International Real-Time Systems Symposium (RTSS)*. IEEE, 2007, pp. 239–243.

[3] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, 1973.

[4] A. Burns and R. I. Davis, "A survey of research into mixed criticality systems," *ACM Computing Surveys*, vol. 50, no. 6, pp. 1–37, 2018.

[5] S. Altmeyer, "What does confidence in wcet estimations depend upon?" in *15th International Workshop on Worst-Case Execution Time Analysis*, 2015, pp. 65–74.

[6] S. Baruah, A. Burns, and R. I. Davis, "Response-time analysis for mixed criticality systems," in *Proceedings of the IEEE Real-Time Systems Symposium*, 2011, pp. 34–43.

[7] S. Baruah and Z. Guo, "Mixed-criticality job models: A comparison," in *Proceedings of the Workshop on Mixed Criticality Systems*, 2015.

[8] P. Ekberg and W. Yi, "Bounding and shaping the demand of mixed-criticality sporadic tasks," in *Proceedings of the 24th Euromicro Conference on Real-Time Systems*. IEEE, 2012, pp. 135–144.

[9] N. Guan, P. Ekberg, M. Stigge, and W. Yi, "Effective and efficient scheduling of certifiable mixed-criticality sporadic task systems," in *Proceedings of the IEEE Real-Time Systems Symposium*. IEEE, 2011, pp. 13–23.